

Solution Architecture

Date	04 July 2026
Team ID	ramanharshitha2005@gmail.com
Project Name	OptiCrop - Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Solution Architecture Diagram:

Provide a high-level visual representation of your solution's architecture. Use the structural layout below as a template to map out your Presentation Layer, Application Layer, and Data Layer. Fill in the specific technologies and components for your project directly into the boxes to create your architectural diagram.

Presentation / Client Layer
index.html (Home) recommend.html (Scenarios 1 & 2) disease.html (Disease Check) insights.html (Research Dashboard — Chart.js) contact.html shared: styles.css, script.js assets: PNG hero images
↓ HTTP Requests (GET / POST) ↑ JSON Responses
Application Layer — server.py (Python ThreadingHTTPServer)
GET static files (safe_resolve) POST /api/recommend POST /api/suitability POST /api/fertilizer POST /api/disease POST /api/image-disease GET /api/weather GET /api/analytics GET /data/opticrop-data.json
↓ joblib.load / model.predict ↑ Prediction results
ML / Inference Layer
crop_model.joblib (RF n=350 + DT + SVC ensemble, LabelEncoder, 9 features) disease_model.joblib (RF n=250, symptom + environment features) image_disease_model.joblib or .h5 (CNN / pixel-feature RF for leaf images)
↓ pd.read_csv / json.load ↑ DataFrames / dicts
Data / Storage Layer
data/Crop_recommendation.csv (2200 rows, 8 cols, 22 crop classes — Kaggle) data/opticrop-data.json (fallback crop profiles + disease records) models/*.joblib models/confusion_matrix.png models/disease_confusion_matrix.png
↓ urllib.request ↑ JSON weather data
External API
Open-Meteo Geocoding API (city → lat/lon) Open-Meteo Forecast API (lat/lon → temperature, humidity, 30-day rainfall) No API key required — fully open and free

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	5 HTML pages + shared CSS/JS. Handles all user interactions: form submission, file upload, live weather fetch, chart rendering, CSV export. Communicates with the server via fetch() API calls.	HTML5, CSS3, Vanilla JavaScript, Chart.js 4.4.1

API Gateway (server.py)	Python ThreadingHTTPServer acts as both static file server and REST API gateway. Routes GET/POST requests to the correct handler. Implements safe_resolve() to prevent directory traversal.	Python http.server, ThreadingHTTPServer, urllib
ML Inference Services	Three ML modules: (1) Crop recommendation ensemble, (2) Disease classification from symptoms+environment, (3) Image disease classification from leaf photo. Each loaded once at request time using joblib.load().	Scikit-learn, Joblib, TensorFlow-CPU (optional)
Data Store	CSV and JSON flat files loaded via Pandas and json.load(). No relational database. Models stored as binary .joblib files. Confusion matrix plots saved as PNG during training.	Pandas, NumPy, CSV, JSON, Joblib
External API	Open-Meteo provides real-time weather (temperature, humidity, 30-day rainfall) using a two-step geocoding + forecast call. No API key, no rate limit for moderate use.	Python urllib.request, Open-Meteo REST API